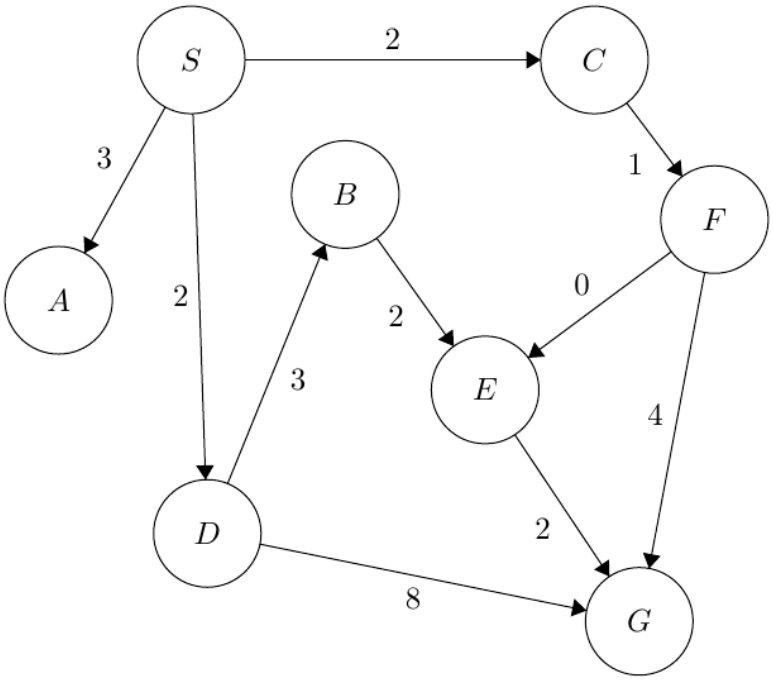
BT01 – TÌM KIẾM TRONG AI

Thông tin	Nội dung
Họ và tên	Tạ Quang Thắng
MSSV / Lớp	3123410345

Bài 1. Các thuật toán tìm kiếm



Hình 1. Đồ thị có hướng trong đề bài (nguồn: BT01, trang 1).

Các cạnh và trọng số: S→A(3), S→C(2), S→D(2), C→F(1), D→B(3), D→G(8), B→E(2), F→E(0), F→G(4), E→G(2).

1.1. Breadth-First Search (BFS)

Bước	Nút lấy ra	Frontier sau mở rộng (đầu → cuối)
1	S	A, C, D
2	A	C, D
3	C	D, F
4	D	F, B, G
5	F	B, G, E
6	B	G, E
7	G (đích)	Kết thúc

Explored Set: S → A → C → D → B → F.

Path: S → D → G.

Total Cost: 2 + 8 = 10.

Giải thích: BFS ưu tiên số cạnh ít nhất. Trong các đường đi từ S tới G, S→D→G chỉ có 2 cạnh nên được tìm trước những đường dài hơn, dù chi phí 10 không phải thấp nhất.

1.2. Depth-First Search (DFS)

Duyệt sâu và chọn nhánh theo alphabet: S→A (ngõ cụt), quay lại S→C→F→E→G. Khi F có hai con E và G, DFS chọn E trước.

Bước	Nút lấy ra	Nhận xét
1	S	Xét A trước C, D
2	A	Ngõ cụt, quay lui
3	C	Đi F
4	F	Ưu tiên E trước G
5	E	Đi G
6	G (đích)	Kết thúc

Explored Set: S → A → C → F → E.

Path: S → C → F → E → G.

Total Cost: 2 + 1 + 0 + 2 = 5.

1.3. Iterative Deepening Search (IDS)

Giới hạn độ sâu	Thứ tự các nút được duyệt	Kết quả
0	S	Chưa gặp G
1	S, A, C, D	Chưa gặp G
2	S, A, C, F, D, B, G	Tìm thấy G ở độ sâu 2

Path cuối cùng: S → D → G.

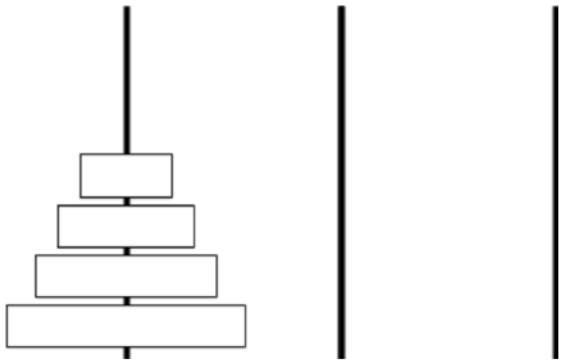
Total Cost: 2 + 8 = 10.

IDS thực hiện DFS giới hạn độ sâu 0, rồi 1, rồi 2. Đích đầu tiên ở độ sâu 2 nằm trên nhánh S→D→G, nên thuật toán dừng tại đó.

Câu hỏi thảo luận: BFS và DFS

BFS sử dụng hàng đợi FIFO, mở rộng các nút theo từng mức độ sâu; nếu mỗi bước có cùng chi phí, BFS tìm được đường đi tối ưu theo số bước. DFS sử dụng ngăn xếp LIFO (hoặc đệ quy), đi sâu trên một nhánh trước khi quay lui; dùng ít bộ nhớ hơn trong nhiều trường hợp nhưng không bảo đảm tìm đường đi ngắn nhất hay có chi phí thấp nhất. Trong đồ thị này, BFS trả về đường chi phí 10 còn DFS trả về đường chi phí 5; điều đó không có nghĩa DFS luôn tốt hơn BFS.

Bài 2. Tháp Hà Nội – Tower of Hanoi



Hình 2. Trạng thái ban đầu của bài toán Tháp Hà Nội (nguồn: BT01, trang 2).

2.1. Biểu diễn trạng thái

Đánh số đĩa từ 1 (nhỏ nhất) đến n (lớn nhất); đánh số ba cọc lần lượt là 1, 2, 3. Biểu diễn một trạng thái bằng bộ n phần tử $s = (p_1, p_2, \ldots, p_n)$, trong đó $p_i \in \{1, 2, 3\}$ cho biết đĩa i đang nằm ở cọc nào. Trên mỗi cọc, thứ tự các đĩa mặc định là đĩa lớn ở dưới, đĩa nhỏ ở trên; vì thế chỉ cần vị trí cọc của từng đĩa là đủ để xác định các nước đi hợp lệ.

2.2. Kích thước không gian trạng thái

Mỗi đĩa có 3 lựa chọn cọc, độc lập về cách chỉ định vị trí. Trên từng cọc, thứ tự hợp lệ được xác định duy nhất theo kích thước đĩa, nên số trạng thái hợp lệ là $|S| = 3^n$. Với hình gồm 4 đĩa, $|S| = 3^4 = 81$ trạng thái. Không tính thêm các hoán vị xếp sai quy tắc "đĩa lớn không nằm trên đĩa nhỏ".

2.3. Trạng thái bắt đầu

$S_0 = (1, 1, \ldots, 1)$, gồm n phần tử: toàn bộ đĩa nằm trên cọc 1. Riêng hình có 4 đĩa: $S_0 = (1, 1, 1, 1)$.

2.4. Các hành động hợp lệ

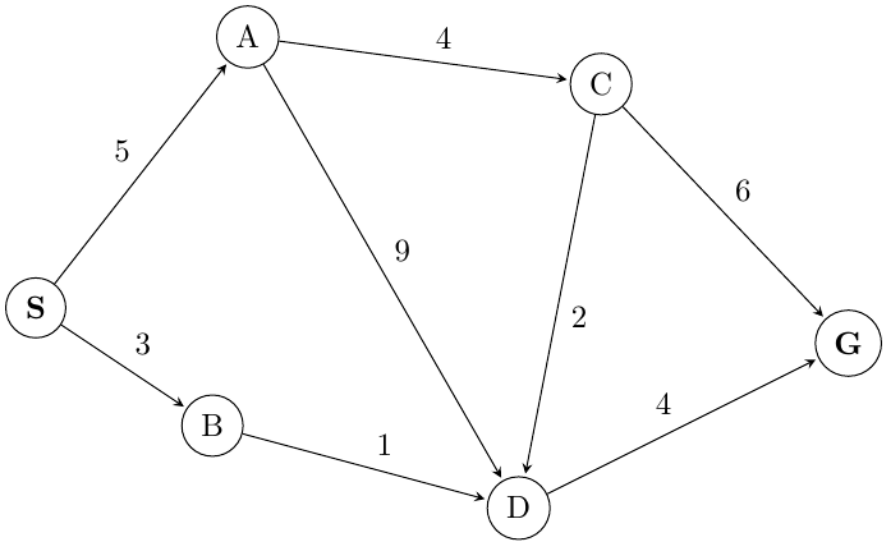
Actions(s) là tập các thao tác Move(i, a, b), nghĩa là chuyển đĩa i từ cọc a sang cọc b (a ≠ b), với các điều kiện: (1) đĩa i đang ở cọc a; (2) đĩa i là đĩa trên cùng cọc a, tức không có đĩa j < i cùng cọc a; (3) cọc b trống hoặc đĩa trên cùng cọc b lớn hơn i, tức không có đĩa j < i trên cọc b. Mỗi hành động chỉ chuyển đúng một đĩa.

Ví dụ với n = 4 từ (1,1,1,1), chỉ đĩa 1 có thể chuyển: Move(1,1,2) → (2,1,1,1) hoặc Move(1,1,3) → (3,1,1,1).

2.5. Goal Test

goalTest(s) trả về True khi và chỉ khi s = (3, 3, …, 3), tức cả n đĩa đều được chuyển sang cọc thứ ba và vẫn xếp hợp lệ. Các trường hợp khác trả về False.

Bài 3. Thiết kế và phân tích Heuristic



Hình 3. Đồ thị dùng để thiết kế heuristic (nguồn: BT01, trang 3).

Khoảng cách tối ưu thực từ từng nút đến G (ký hiệu h*) tính theo các cạnh có hướng: h*(G)=0; h*(D)=4; h*(C)=min{6,2+4}=6; h*(B)=1+4=5; h*(A)=min{4+6,9+4}=10; h*(S)=min{5+10,3+5}=8.

3.1. Một heuristic vừa admissible vừa consistent

Nút n	**h*(n) – chi phí tối ưu**	**h(n) đề xuất**
S	8	7
A	10	9
B	5	4

C	6	5
D	4	3
G	0	0

Kiểm tra admissible: với mọi n, $0 \le h(n) \le h^*(n)$; vậy heuristic không đánh giá quá cao chi phí tới đích.

Kiểm tra consistent trên các cạnh: S→A: 7≤5+9; S→B: 7≤3+4; A→C: 9≤4+5; A→D: 9≤9+3; B→D: 4≤1+3; C→D: 5≤2+3; C→G: 5≤6+0; D→G: 3≤4+0. Tất cả đều đúng.

3.2. Chạy A* với heuristic đã thiết kế

A* dùng f(n)=g(n)+h(n). Bảng dưới thể hiện diễn biến lấy nút có f nhỏ nhất; khi hòa mới xét alphabet.

Bước	**Nút lấy ra**	**g(n)**	**h(n)**	**f(n)**	**Thao tác**
1	S	0	7	7	Thêm A: (g=5,f=14), B: (g=3,f=7)
2	B	3	4	7	Thêm D: (g=4,f=7)
3	D	4	3	7	Thêm G: (g=8,f=8)
4	G (đích)	8	0	8	Dừng

Explored Set: S → B → D.

Path: S → B → D → G.

Total Cost: 3 + 1 + 4 = 8.

3.3. Heuristic admissible nhưng không consistent

Nút	**S**	**A**	**B**	**C**	**D**	**G**
h(n)	8	10	0	6	4	0

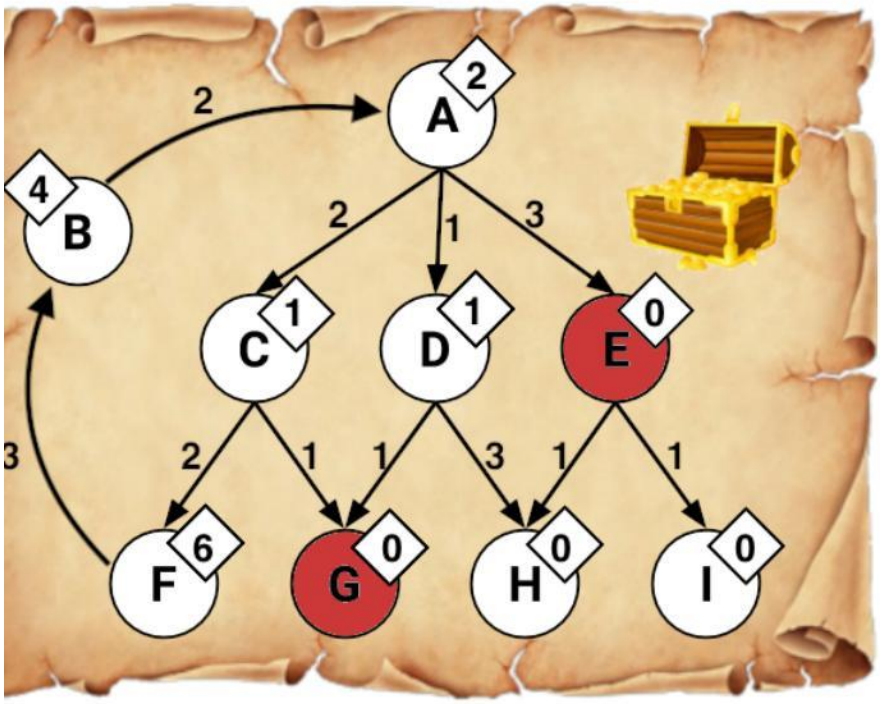
Bộ giá trị trên vẫn admissible vì $h(n) \le h^*(n)$ tại mọi nút. Tuy nhiên cạnh S→B có c(S,B)=3 và h(S)=8 > 3 + h(B)=3+0=3. Vậy điều kiện nhất quán bị vi phạm; heuristic không consistent.

3.4. Vì sao consistent suy ra admissible?

Với mọi cạnh (u,v), tính consistent cho ta $h(u) \le c(u,v)+h(v)$. Xét một đường đi tối ưu từ nút n tới đích G: $n=n_0 \to n_1 \to \ldots \to n_k=G$. Áp dụng bất đẳng thức liên tiếp rồi cộng vế theo vế, ta được $h(n) \le c(n_0,n_1)+\ldots+c(n_{k-1},G)+h(G)$. Vì h(G)=0 nên $h(n) \le h^*(n)$. Do đó h là

admissible. Kết luận dùng cho các nút có đường đi tới G; với nút không đến được đích thì h*(n)=+∞.

Bài 4. Treasure Hunting (Tìm kho báu)



Hình 4. Bản đồ tìm kho báu giữ nguyên từ đề (nguồn: BT01, trang 5).

Các nút đích màu đỏ: E và G. Điểm xuất phát: A. Các cạnh: A→C(2), A→D(1), A→E(3), B→A(2), C→F(2), C→G(1), D→G(1), D→H(3), E→H(1), E→I(1), F→B(3). Heuristic từ hình: h(A)=2, h(B)=4, h(C)=1, h(D)=1, h(E)=0, h(F)=6, h(G)=0, h(H)=0, h(I)=0.

4.1–4.5. Kết quả từng thuật toán

Thuật toán	Explored Set (thứ tự)	Path returned	Cost
DFS	A → C → F → B	A → C → G	2 + 1 = 3
BFS	A → C → D	A → E	3
UCS	A → D → C	A → D → G	1 + 1 = 2
Greedy	A	A → E	3
A*	A → D	A → D → G	1 + 1 = 2

DFS – tìm kiếm theo chiều sâu

A→C vì C đứng trước D, E. Tại C xét F trước G; từ F đi B; B dẫn về A đã được explored nên quay lui. Sau đó xét G và kết thúc. Explored = [A,C,F,B]; đường A→C→G, chi phí 3.

BFS – tìm kiếm theo chiều rộng

Mở A, tạo frontier [C,D,E]; mở C và tạo F,G; mở D và có thể tạo H (G đã có trong frontier); kế tiếp E nằm ở độ sâu 1 nên được lấy ra và kết thúc. Explored = [A,C,D]; đường A→E có 1 bước, chi phí 3.

UCS – chi phí tích lũy nhỏ nhất

Lấy A(g=0) → frontier D(1), C(2), E(3). Lấy D(g=1) → thêm G(2), H(4). Ở mức g=2, C và G hòa nên lấy C trước (theo alphabet). Từ C, lối C→G cho g=3 không cải thiện G đang có g=2. Cuối cùng lấy G(g=2), dừng. Explored = [A,D,C]; đường A→D→G, chi phí 2.

Greedy – ưu tiên h(n) nhỏ nhất

Từ A, các con C(h=1), D(h=1), E(h=0); chọn E ngay vì h(E)=0 và E là đích. Explored = [A]; đường A→E, chi phí 3. Greedy không xét trực tiếp chi phí g(n) nên chưa tối ưu ở ví dụ này.

A* – ưu tiên f(n)=g(n)+h(n)

Lấy A(f=2). Frontier: D(g=1,h=1,f=2), C(g=2,h=1,f=3), E(g=3,h=0,f=3). Lấy D(f=2) → sinh G(g=2,h=0,f=2), H(g=4,h=0,f=4). Lấy G(f=2) và dừng. Explored = [A,D]; đường A→D→G, chi phí 2.

Bảng tổng hợp so sánh

Thuật toán	Tiêu chí ưu tiên	Đường đi tìm được	Chi phí
DFS	Độ sâu lớn nhất	A → C → G	3
BFS	Độ sâu nhỏ nhất	A → E	3
UCS	g(n) nhỏ nhất	A → D → G	2
Greedy	h(n) nhỏ nhất	A → E	3
A*	g(n)+h(n) nhỏ nhất	A → D → G	2

Nhận xét: UCS và A* tìm được đường đi tối ưu chi phí 2; BFS và Greedy đi trực tiếp đến E nhưng chi phí 3. DFS ở ví dụ này tìm G qua C với chi phí 3.

Bài 5. Câu hỏi Đúng/Sai

Câu 1. DFS luôn mở rộng ít nhất nhiều nút bằng A* có heuristic admissible.

Đáp án: Sai.

DFS có thể may mắn đi thẳng đến đích theo nhánh được chọn đầu tiên, còn A* vẫn cần mở rộng các nút có f(n) thấp hơn chi phí nghiệm. Vì thế không có bảo đảm chung DFS luôn mở rộng nhiều nút hơn hoặc bằng A*.

Câu 2. Manhattan distance là admissible cho số nước đi tối thiểu của quân xe.

Đáp án: Sai.

Quân xe có thể di chuyển nhiều ô cùng một hàng hoặc cột trong một nước. Ví dụ từ a1 đến a8 trên bàn trống: số nước tối ưu bằng 1, nhưng Manhattan distance = 7 > 1. Như vậy heuristic đánh giá quá cao.

Câu 3. Euclidean distance là admissible cho tìm đường của Pacman.

Đáp án: Đúng (với lưới Pacman thông thường, mỗi bước đi sang ô kề theo 4 hướng có chi phí 1).

Khoảng cách đường thẳng không lớn hơn số bước Manhattan và số bước đi thực tế khi phải tránh tường: $d_{Euclidean} \leq d_{Manhattan} \leq h^*$. Vì thế nó không đánh giá quá cao. Nếu biến thể Pacman có dịch chuyển tức thời, cổng dịch chuyển hay chi phí bất thường thì phải kiểm tra lại giả định này.

Câu 4. Tổng các heuristic admissible luôn admissible.

Đáp án: Sai.

Giả sử chi phí còn lại tối ưu $h^*(n)=5$ và có hai heuristic admissible $h_1(n)=4$, $h_2(n)=4$. Khi cộng: $h_1+h_2=8 > 5$, nên tổng không admissible. Chỉ có thể cộng an toàn khi chứng minh các thành phần ước lượng những phần chi phí không trùng lặp (hoặc theo một điều kiện đủ thích hợp).

Câu 5. Admissibility kéo theo consistency.

Đáp án: Sai.

Một heuristic có thể không vượt chi phí tối ưu ở mỗi nút nhưng vẫn vi phạm bất đẳng thức trên một cạnh. Phản ví dụ ngay Bài 3.3: $h(S)=8 \leq h^*(S)=8$, $h(B)=0 \leq h^*(B)=5$, song $8>3+0$ trên cạnh $S \rightarrow B$. Consistency suy ra admissibility (khi $h(G)=0$), chiều ngược lại không đúng.

Câu 6. A* graph search luôn tối ưu nếu heuristic admissible.

Đáp án: Sai nếu graph search không mở lại các nút đã đóng; admissible một mình chưa đủ để bảo đảm tính tối ưu trong biến thể đó.

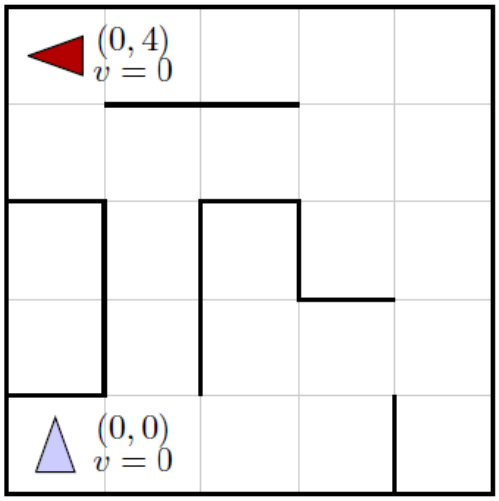
Phản ví dụ: S→A(2), S→B(1), B→A(0,5), A→G(2), B→G(100). Chọn h(S)=0, h(A)=0, h(B)=2,5, h(G)=0. Tất cả giá trị đều admissible, nhưng h(B)=2,5 > 0,5+h(A)=0,5 nên không consistent. A* không reopen có thể đóng A theo g=2 trước, rồi bỏ qua đường B→A có g=1,5 và trả về S→A→G chi phí 4 thay vì đường tối ưu S→B→A→G chi phí 3,5. Nếu cho phép cập nhật và mở lại nút khi tìm được đường tốt hơn, A* vẫn có thể bảo đảm tối ưu dưới các giả định chuẩn; hoặc dùng heuristic consistent để bảo đảm tối ưu với graph search không cần reopen.

Tổng kết

Bài	Nội dung	Kết quả trọng tâm
1	Tìm kiếm BFS / DFS / IDS	BFS: S-D-G, 10; DFS: S-C-F-E-G, 5; IDS: S-D-G, 10
2	Tower of Hanoi	Biểu diễn $(p_1,\ldots,p_n)$; số trạng thái 3^n; đích (3,…,3)
3	Thiết kế heuristic và A*	h=(7,9,4,5,3,0) theo S,A,B,C,D,G; A* S-B-D-G, 8
4	Treasure Hunting	UCS & A*: A-D-G, 2; BFS & Greedy: A-E, 3; DFS: A-C-G, 3
5	Đúng/Sai	1 Sai; 2 Sai; 3 Đúng*; 4 Sai; 5 Sai; 6 Sai**

BT01b
TÌM KIẾM VÀ HEURISTIC TRONG AI

Câu 1. Tìm kiếm và heuristic



Hình 1. Mê cung 5 × 5 và trạng thái xuất phát/đích (trích nguyên hình từ đề).

Trạng thái được biểu diễn bởi s = (x, y, hướng, v). Một lần tăng/giữ/giảm tốc cập nhật vận tốc trước, sau đó di chuyển v′ ô; khi v′ = 0, hành động giảm tốc không dịch chuyển. Rẽ chỉ hợp lệ lúc v = 0. Đích đầy đủ là (0, 4, Tây, 0).

a) Chuỗi hành động ngắn nhất qua nhánh trái (1,2)

Đường đi theo vị trí: (0,0) → (1,0) → (1,1) → (1,2) → (1,3) → (0,3) → (0,4). Xe phải dừng để rẽ tại các vị trí (1,0), (1,3), (0,3) và phải dừng đúng hướng Tây ở ô cuối.

Bước	Hành động	Trạng thái sau hành động
0	Bắt đầu	(0,0, B, 0)
1	Rẽ phải	(0,0, Đ, 0)
2	Tăng tốc	(1,0, Đ, 1)
3	Giảm tốc	(1,0, Đ, 0)
4	Rẽ trái	(1,0, B, 0)
5	Tăng tốc	(1,1, B, 1)
6	Giữ tốc độ	(1,2, B, 1)
7	Giữ tốc độ	(1,3, B, 1)
8	Giảm tốc	(1,3, B, 0)
9	Rẽ trái	(1,3, T, 0)
10	Tăng tốc	(0,3, T, 1)
11	Giảm tốc	(0,3, T, 0)
12	Rẽ phải	(0,3, B, 0)
13	Tăng tốc	(0,4, B, 1)
14	Giảm tốc	(0,4, B, 0)
15	Rẽ trái	(0,4, T, 0)

Đáp án: Rẽ phải → Tăng tốc → Giảm tốc → Rẽ trái → Tăng tốc → Giữ tốc độ → Giữ tốc độ → Giảm tốc → Rẽ trái → Tăng tốc → Giảm tốc → Rẽ phải → Tăng tốc → Giảm tốc → Rẽ trái. Tổng cộng 15 hành động.

Kiểm tra số bước: Các đoạn tiến sau khi dừng gồm 1, 3, 1 và 1 ô; số hành động để đi và dừng tương ứng là 2 + 4 + 2 + 2 = 10. Có 5 thao tác rẽ. Tổng = 10 + 5 = 15. Mô phỏng BFS trên toàn bộ trạng thái với Vmax ≥ 2 xác nhận không có phương án ngắn hơn; phương án này có đi qua (1,2).

b) Kích thước không gian trạng thái

Mỗi trạng thái chứa: M × N khả năng chọn vị trí, 4 khả năng chọn hướng và Vmax + 1 giá trị vận tốc nguyên (0,1,…,Vmax). Do đó:

Đáp án: $|S| = 4MN(V_{max} + 1)$.

Công thức đúng dưới giả thiết của đề rằng mọi cấu hình (vị trí, hướng, vận tốc) đều có thể đạt được.

c) Có thể có trạng thái không có nút con không?

Đáp án: Có, khi Vmax ≥ 2.

Ví dụ s = (1,3, Bắc, 2). Ngay phía Bắc là tường; rẽ trái/phải không hợp lệ vì $v \neq 0$; Giữ tốc độ ($v'=2$) hoặc Tăng tốc ($v' \geq 3$) đều va tường; Giảm tốc còn $v'=1$ nên vẫn va tường. Vì vậy Succ(s) = ∅.

> **Lưu ý.** Nếu Vmax = 1 thì mọi trạng thái với v = 1 đều giảm xuống $v' = 0$ mà không di chuyển được; trạng thái v = 0 luôn rẽ được. Trường hợp đó không có nút cụt theo các luật đã cho.

d) Hệ số phân nhánh lớn nhất

Đáp án: bmax = 3 (giả sử Vmax ≥ 1).

Với v = 0, tối đa ba hành động hợp lệ là Rẽ trái, Rẽ phải và Tăng tốc. Với v > 0, tối đa ba hành động là Tăng tốc, Giữ tốc độ và Giảm tốc. Ví dụ (0,0, Bắc, 0) đạt 3 hành động hợp lệ là sai, vì có tường giữa (0,0) và (0,1). Hành động Tăng tốc sẽ va tường. Có thể thay bằng (1,0, Bắc, 0): Tăng tốc, rẽ trái và rẽ phải đều hợp lệ.

e) Manhattan distance có admissible không?

Đặt $h_M(s) = |x - 0| + |y - 4|$. Phải phân biệt rõ hai cách hiểu của câu hỏi: đánh giá công thức cho mô hình xe trên lưới bất kỳ, hay chỉ đánh giá đúng mê cung 5 × 5 được vẽ.

Kết luận theo mô hình tổng quát: Không. Một hành động có thể đi qua nhiều ô, nên khoảng cách Manhattan có thể lớn hơn số hành động tối ưu.

Phản ví dụ trên lưới có hành lang thẳng thông suốt ít nhất 6 ô, Vmax ≥ 2, cùng đích (0,4,Tây,0): bắt đầu ở (6,4,Tây,0). Khi đó hM = 6. Thực hiện Tăng tốc (đi 1 ô), Tăng tốc (đi 2 ô), Giữ tốc độ (đi 2 ô), Giảm tốc (đi 1 ô), Giảm tốc (đi 0 ô) sẽ tới đích trong 5 hành động. Với 4 hành động từ v = 0 và phải kết thúc v = 0, quãng đường đi tối đa là 1 + 2 + 1 + 0 = 4 ô, nên h* = 5. Do 6 > 5, hM không admissible trong họ bài toán tổng quát.

> **Lưu ý.** Điểm cần chú ý khi chấm đúng hình đề: (6,4) nằm ngoài mê cung 5 × 5. Trên chính mê cung được vẽ, kiểm tra toàn bộ trạng thái có đường tới đích với v ≤ 4 không tìm được trạng thái nào có hM > h*. Vì vậy nếu yêu cầu phản ví dụ phải nằm trong đúng mê cung 5 × 5 và đích phải đúng hướng/vận tốc, nên kết luận "Có" cho riêng trường hợp hữu hạn ấy. Đề chưa nói rõ phạm vi đánh giá, nên không thể dùng phản ví dụ ngoài hình để khẳng định riêng hình bị sai.

f) h(s) = Manhattan distance / Vmax có admissible không?

Đáp án: Có, với Vmax ≥ 1.

Mỗi hành động chỉ di chuyển tối đa Vmax ô theo một hướng; hành động rẽ di chuyển 0 ô. Nếu lời giải cần k hành động, tổng quãng đường đi không nhỏ hơn khoảng cách Manhattan D nhưng không vượt k·Vmax. Suy ra k ≥ D/Vmax và h*(s) ≥ D/Vmax. Heuristic này là một cận dưới, nên admissible.

g) Heuristic không admissible có ảnh hưởng tính đầy đủ của A* Tree Search?

Đáp án: Không, với giả thiết bài toán đã nêu và chi phí mỗi hành động bằng 1.

Mặc dù đồ thị trạng thái hữu hạn có thể tạo ra cây tìm kiếm vô hạn do chu trình, b hữu hạn và mỗi bước tăng g lên 1. Heuristic không âm và hữu hạn trên tập trạng thái hữu hạn; vì thế chỉ có hữu hạn nút có f(n) không lớn hơn f của một nút đích hữu hạn đã có đường đi. A* cuối cùng sẽ lấy nút đích ra khỏi frontier nếu đích đạt được.

h) Heuristic không admissible có ảnh hưởng tính tối ưu của A* Tree Search?

Đáp án: Có.

Nếu h đánh giá quá cao trên nhánh tối ưu, các nút của nhánh này có f = g + h lớn hơn nút đích thuộc nhánh dài hơn. A* có thể chọn đích không tối ưu trước; bảo đảm tối ưu của A* Tree Search không còn khi mất tính admissible.

i) Lý do hợp lý để sử dụng heuristic không admissible

Phương án	Chọn	Giải thích ngắn

1. Dễ tính hơn	Có	Tiết kiệm thời gian ước lượng heuristic.
2. Gần chi phí thực hơn; mở ít nút hơn	Có	Đánh đổi bảo đảm tối ưu lấy tốc độ.
3. Luôn tối ưu khi chi phí thực không âm	Không	Không có bảo đảm tối ưu.
4. Không xét một phần đồ thị	Có, theo nghĩa thực dụng	Có thể dừng trước khi mở rộng nhiều vùng; không phải tự động xóa/khóa các vùng này.

Đáp án: Chọn 1, 2 và 4 (hiểu 4 là "không cần mở rộng trước khi dừng", không phải loại bỏ tuyệt đối mọi nút).

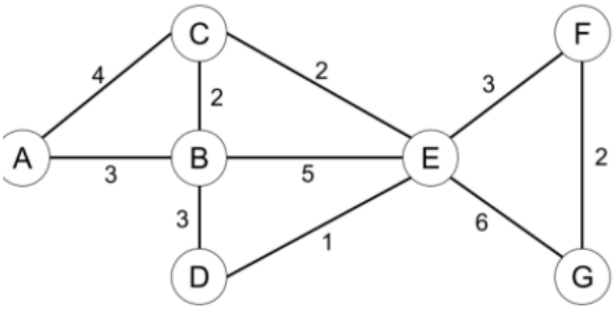
Câu 2. Các nút tìm kiếm

Đề dùng Tree Search, không có explored set. Gọi d(ns) là độ sâu của nghiệm nông nhất, C* = g(nc) là chi phí tối ưu; f(n) = g(n) + h(n). Bỏ qua trường hợp hòa đúng như yêu cầu.

Thuật toán	**Bất đẳng thức cho mọi nút mở rộng trước khi dừng**	**Giải thích**
a) BFS	d(n) < d(ns)	BFS ưu tiên độ sâu, nên xử lý các nút nông hơn nút đích nông nhất.
b) UCS	g(n) < g(nc) = C*	UCS ưu tiên tổng chi phí đường đi đã đi.
c) A* (h consistent)	f(n) = g(n) + h(n) < C*	A* ưu tiên f; tại goal h = 0, giá trị f của nghiệm tối ưu là C*.

Lưu ý. Đây là các điều kiện ưu tiên nghiêm ngặt, không xét nút đồng hạng. Để mô tả tất cả nút có thể được mở rộng trước khi dừng, phải thêm yếu tố phá hòa và cách kiểm tra đích; bài yêu cầu bỏ qua trường hợp đó.

Câu 3. Tìm kiếm trên đồ thị



Node	h_1	h_2
A	12	11
B	6	7
C	9	6
D	3	4
E	3	5
F	2	1
G	0	0

Hình 2. Đồ thị vô hướng có trọng số cùng hai heuristic h1, h2 (trích từ đề).

Các cạnh: A–B:3, A–C:4, B–C:2, B–D:3, B–E:5, C–E:2, D–E:1, E–F:3, E–G:6, F–G:2. Đích G, bắt đầu A.

a) Đường đi thuật toán có thể trả về

Thuật toán	A–C–E–G	A–C–E–F–G	A–B–D–E–F–G	Khác
UCS		✓		
Greedy với h1				✓
Greedy với h2	✓			
A* với h1			✓	
A* với h2		✓		

Ở hàng Greedy h1, đường đi thuộc cột "Khác" là A–B–E–G (chi phí 14) hoặc A–B–D–E–G (chi phí 13), tùy phá hòa tại B và chính sách cập nhật đường cha khi E đã có trên frontier. Chúng đều kết thúc qua cạnh E–G thay vì E–F–G do h1(G) = 0 < h1(F) = 2.

Giải thích quá trình lựa chọn

- UCS: A(g=0) → B(g=3) → C(g=4) → E(g=6) → F(g=9) → G(g=11); có thể có D(g=6) xen vào do hòa. Đường về A–C–E–F–G.
- Greedy h1: từ A ưu tiên B vì h1(B)=6 < h1(C)=9; tại B, D và E cùng h1=3; khi đến E, ưu tiên G (h1=0) trước F (h1=2).
- Greedy h2: A → C → E → G, bởi h2(C)=6 < h2(B)=7, rồi h2(E)=5 và h2(G)=0.
- A* h1: A(f=12) → B(f=9) → D(f=9) → E(f=10) → F(f=12) → G(f=12). C vẫn có f=13 nên A* dừng với đường A–B–D–E–F–G.
- A* h2: B và C cùng f=10 tại bước đầu; sau các cập nhật đường tốt hơn, kết quả trong đồ thị này là A–C–E–F–G, g(G)=11.

b) Chi phí đường đi tối ưu theo UCS

Đáp án: A → C → E → F → G; C* = 4 + 2 + 3 + 2 = 11.

Hai đường được liệt kê còn lại có chi phí: A–C–E–G = 4+2+6 = 12; A–B–D–E–F–G = 3+3+1+3+2 = 12. Vì UCS luôn chọn g nhỏ nhất trên đồ thị có chi phí dương, chi phí tối ưu là 11.

c) h1 admissible? consistent?

Admissible: Không. h1(A)=12 > h*(A)=11; h1(C)=9 > h*(C)=7.

Consistent: Không. Trên cạnh A–B: h1(A)=12 > 3+h1(B)=9; cạnh C–E: h1(C)=9 > 2+h1(E)=5.

d) h2 admissible? consistent?

Nút	A	B	C	D	E	F	G
h*(n)	11	9	7	6	5	2	0
h2(n)	11	7	6	4	5	1	0

Admissible: Có, vì $h2(n) \leq h^*(n)$ với mọi nút.

Consistent: Không. Trên cạnh A–B: $11 > 3+7 = 10$; trên cạnh E–F: $5 > 3+1 = 4$.

Câu 4. Trắc nghiệm và trả lời ngắn về tìm kiếm

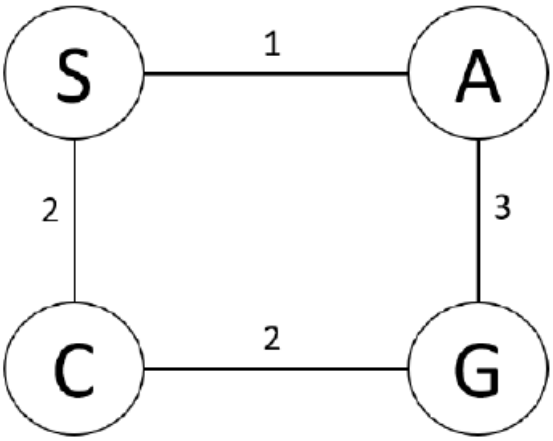
a) Đánh dấu Đúng/Sai

STT	Phát biểu (rút gọn)	Đáp án
1	DFS Tree Search trên đồ thị hữu hạn luôn complete.	Sai
2	BFS Tree Search trên đồ thị hữu hạn luôn complete.	Đúng
3	Iterative Deepening Tree Search trên đồ thị hữu hạn luôn complete.	Đúng
4	Đồ thị không chu trình thì Graph Search luôn có frontier lớn hơn Tree Search.	Sai
5	Iterative Deepening Graph Search có time BFS và memory DFS.	Sai
6	min(h1,h2) luôn consistent nếu cả hai consistent.	Đúng

Giải thích:

- 1. Sai: DFS Tree Search có thể lặp vô hạn trên chu trình S→A→S→A… và không tìm đến nhánh có đích, dù số trạng thái hữu hạn.
- 2. Đúng: Với branching hữu hạn, BFS kiểm tra dần từng độ sâu; nếu có nghiệm ở độ sâu $d < \infty$, BFS sẽ đến được nghiệm ấy.
- 3. Đúng: IDS lặp giới hạn độ sâu 0,1,2,…; khi giới hạn bằng độ sâu nghiệm, depth-limited tree search có thể tìm ra đích.
- 4. Sai: Graph Search loại bỏ trạng thái trùng, do đó có thể giảm chứ không bắt buộc tăng số nút frontier; trên cây không có hợp nhất, frontier có thể bằng nhau.
- 5. Sai: IDS kiểu Tree Search dùng bộ nhớ theo độ sâu, còn Graph Search cần lưu explored set ở từng lượt nên không có bảo đảm bộ nhớ O(bd) của DFS. Thời gian cũng không mặc nhiên trùng với BFS trên mọi đồ thị.
- 6. Đúng: Từ $hi(u) \leq c(u,v)+hi(v)$ với i=1,2, suy ra $\min\{h1(u),h2(u)\} \leq c(u,v)+\min\{h1(v),h2(v)\}$. Giá trị tại đích bằng 0 nếu từng hi tại đích bằng 0.

b) Điền heuristic admissible nhưng không consistent



Hình 3. Đồ thị S–A–G và S–C–G (trích nguyên hình từ đề).

Chi phí tối ưu thật tới G: h*(G)=0, h*(A)=3, h*(C)=2, h*(S)=min(1+3, 2+2)=4.

Trạng thái	**S**	**A**	**C**	**G**
h*(n)	4	3	2	0
h(n) đề xuất	4	0	2	0

Kiểm tra admissible: $4 \le 4$ (S), $0 \le 3$ (A), $2 \le 2$ (C), $0 \le 0$ (G). Tất cả đều thỏa mãn $h(n) \le h^*(n)$.

Kiểm tra consistency tại cạnh S–A có c(S,A)=1: $h(S)=4 > 1+h(A)=1$. Bất đẳng thức $h(S) \le c(S,A)+h(A)$ bị vi phạm, nên h admissible nhưng không consistent.

Đáp án cuối: h(S)=4, h(A)=0, h(C)=2, h(G)=0.

— HẾT —